

CS339: Abstractions and Paradigms for Programming

Stream Processing

Manas Thakur
CSE, IIT Bombay

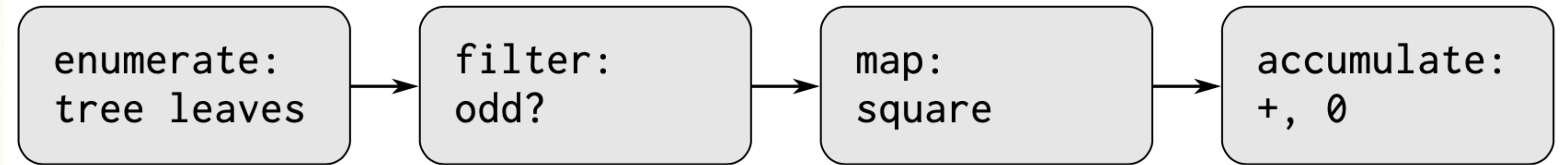


Autumn 2026

Recall from Lecture 11

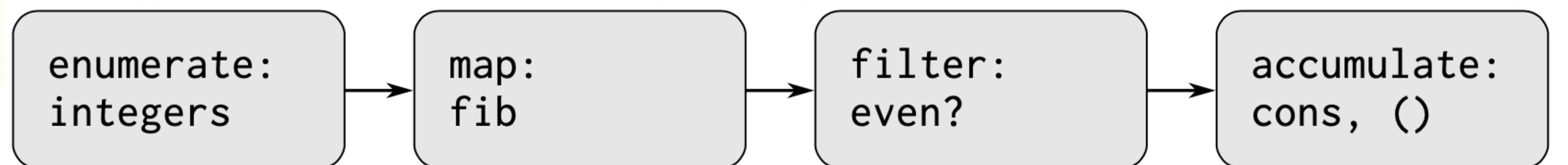


Sum the squares of the odd leaves of a tree



```
(define (sum-odd-squares tree)
  (accumulate
    + 0 (map square (filter odd? (enumerate-tree tree)))))
```

List the even fibonacci numbers till $fib(n)$



```
(define (even-fibs n)
  (accumulate
    cons
    nil
    (filter even? (map fib (enumerate-interval 0 n)))))
```



A PL for specifying computations using sequences

- We had created a new **programming language** that defined *sequences as conventional interfaces*:
 - Sequences (implemented using lists) are like signals flowing from one stage to another
 - `map`, `filter` and `accumulate` (the new vocabulary of our language) represent general patterns of processing sequences
 - **Advantage:** Largely decoupled **modules** defined in terms of operations on sequences



Modularity in full glow

- List the squares of the first $n+1$ fibonacci numbers:

```
(define (list-fib-squares n)
  (accumulate
    cons
    nil
    (map square (map fib (enumerate-interval 0 n))))))
```

- Multiply the squares of the odd integers in a sequence:

```
(define (product-of-squares-of-odd-elements sequence)
  (accumulate * 1 (map square (filter odd? sequence))))
```

- Can you see a BIG problem with our elegant modular programs?



Checking if a number is prime

- Find the smallest divisor (>1) of a given number n
- If the divisor is the same as n , then n is prime

```
(define (smallest-divisor n) (find-divisor n 2))

(define (find-divisor n test-divisor)
  (cond ((> (square test-divisor) n) n)
        ((divides? test-divisor n) test-divisor)
        (else (find-divisor n (+ test-divisor 1)))))

(define (square x) (* x x))

(define (divides? a b) (= (remainder b a) 0))

(define (prime? n)
  (= n (smallest-divisor n)))
```



Second prime between 10000 and 1000000

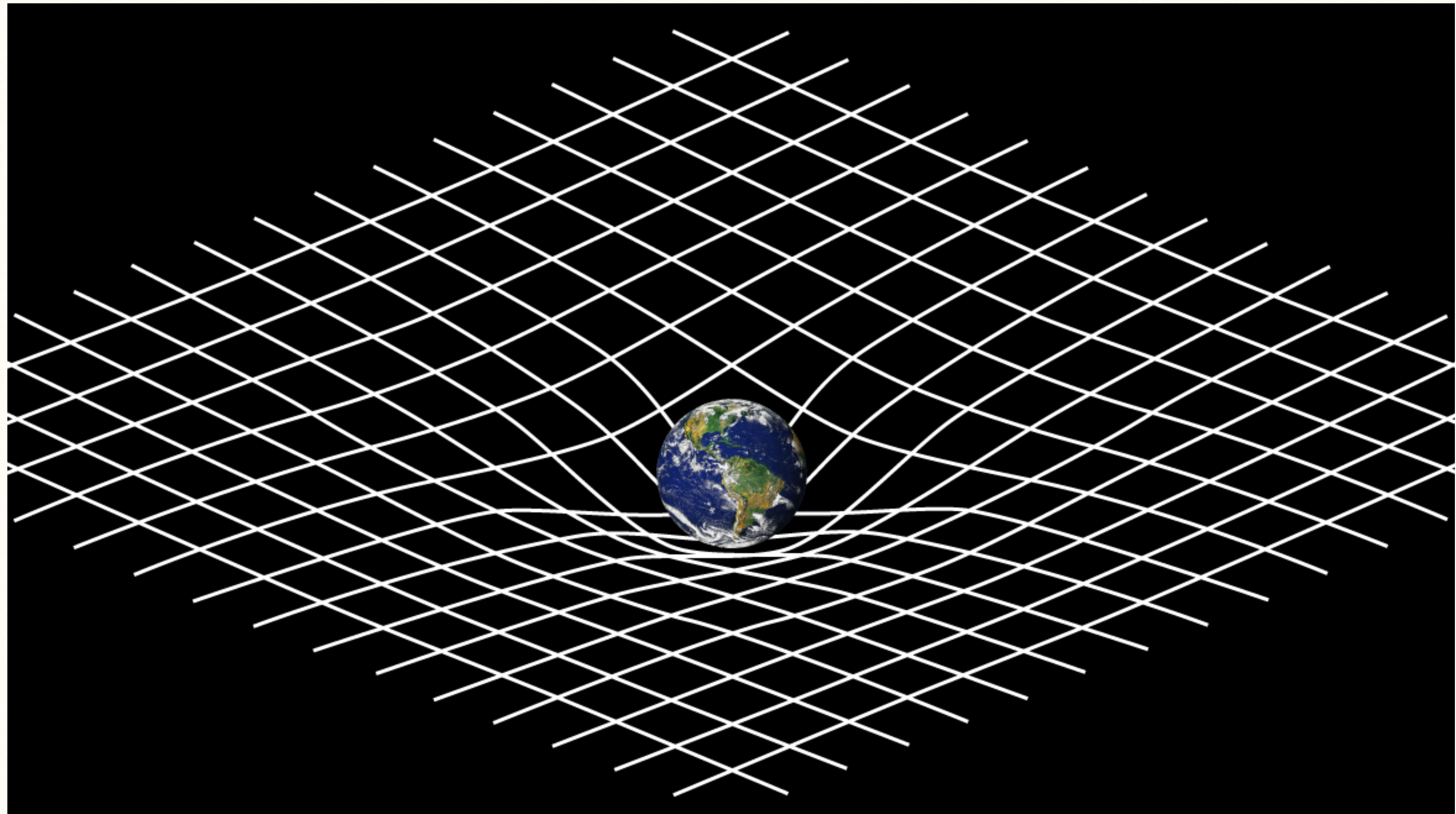
```
(car (cdr (filter prime?
              (enumerate-interval 10000 1000000))))
```

➤ **Very very inefficient:**

- We are enumerating (and storing) all the numbers.
 - We are checking all the numbers for primeness.
 - Whereas we need to return only the second prime number.
 - A messy loopy program could avoid all of that, but, it is messy!
- We would solve this problem now and get elegant programs that are efficient, as well as another view of the imperative world, with **Stream Processing**.



The Einstein View of an OO World



Streams as a Data Structure

➤ Constructor:

```
(cons-stream a b) ==> stream say 's'
```

➤ Selectors:

```
(stream-car s) ==> a
```

```
(stream-cdr s) ==> b
```

➤ Utilities:

```
(the-empty-stream) ==> '()
```

```
(stream-null? s) ==> (eq? s 'the-empty-stream)
```



Streams as Conventional Interfaces

```
(define (stream-ei low high)
  (if (> low high)
      the-empty-stream
      (cons-stream low (stream-ei (+ low 1) high))))
```

**But now we must
get suspicious!**

```
(define (stream-map f s)
  (if (stream-null? s)
      the-empty-stream
      (cons-stream (f (stream-car s)) (stream-map f (stream-cdr s)))))
```

```
(define (stream-filter pred s)
  (cond ((stream-null? s) the-empty-stream)
        ((pred (stream-car s)) (cons-stream (stream-car s)
                                              (stream-filter pred (stream-cdr s))))
        (else (stream-filter pred (stream-cdr s)))))
```

```
(stream-car (stream-cdr (stream-filter prime?
                                          (stream-enumerate-interval 10000 1000000))))
```



Streams are *Delayed Lists*

- What they really are:

```
(cons-stream a b) ==> (cons a (delay b))
```

```
(stream-car s) ==> (car s)
```

```
(stream-cdr s) ==> (force (cdr s))
```

```
(delay <expr>) ==> (lambda() <expr>)
```

```
(force <promise>) ==> (<promise>)
```



Now let's work out

```
(stream-car (stream-cdr (stream-filter prime?  
                        (stream-enumerate-interval 10000 1000000))))
```

Simple substitution:

```
(cons 10000 (delay (stream-enumerate-interval 10001 1000000)))
```

From the else block of `stream-filter`, we have a `stream-cdr` that forces the promise:

```
(cons 10001 (delay (stream-enumerate-interval 10002 1000000)))
```

This continues until `stream-filter`'s predicate evaluates to `true` for 10007:

```
(cons 10007 (delay (stream-enumerate-interval 10008 1000000)))
```



Now let's work out (Cont.)

```
(cons 10007 (delay (stream-enumerate-interval 10008 1000000)))
```

The consequent in `stream-filter` results into:

```
(cons 10007 (delay (stream-filter prime?
                                (cons 10008
                                      (delay (stream-enumerate-interval 10009
                                                                          1000000)))))))
```

Now the outer `stream-cdr` comes into action and forces promises to find the next prime:

```
(cons 10009 (delay (stream-filter prime?
                                (cons 10010
                                      (delay (stream-enumerate-interval 10011
                                                                          1000000)))))))
```

Returned by **stream-car**



What is really happening?

- Is `stream-enumerator` pushing more of the stream to `stream-filter`? Or
- Is `stream-filter` pulling more of the stream from `stream-enumerator`?
- The stream is being **generated on demand**:
 - `stream-car` wants one thing from `stream-cdr`, which wants two things from `stream-filter`, which wants enough of the stream to be generated by `stream-enumerator`
- Thus, we are back to the space and time efficiency of the loopy program, without it looking messy! 🎉

<https://www.cse.iitb.ac.in/~manas/teaching/stream-anims/>



Unevaluated...



Post midsem: we experience a new level of *expressiveness* — the kind that comes when our PL allows us to work with *things that don't exist*.